

SIH PROJECT REPORT
ON
LINGUABOT: A MULTILINGUAL CAMPUS CHATBOT FOR
EQUITABLE
STUDENT INFORMATION ACCESS

Submitted in Partial Fulfillment of the Requirements for the Degree of
Bachelor of Engineering

in
Electronics and Communication Engineering

Department of Electronics and Communication Engineering

VSB ENGINEERING COLLEGE

Karur, Tamil Nadu

Academic Year: 2025–2026

MAYUKA M
(922525106195)

ABSTRACT

The proliferation of digital communication has fundamentally altered the expectations of students with respect to the accessibility and immediacy of institutional information. Campus offices in engineering colleges across India are routinely inundated with repetitive enquiries relating to fee deadlines, scholarship application procedures, examination schedules, and certificate issuance processes. This situation is further complicated by the linguistic diversity of the student population, a significant proportion of whom are more comfortable articulating their queries in Tamil, Hindi, or other regional languages than in English. The resulting communication gap between students and administrative personnel gives rise to long queues at office counters, misunderstood instructions, missed deadlines, and a general erosion of the quality of the institutional experience.

This project presents the design, implementation, and evaluation of LinguaBot, a multilingual conversational campus assistant developed as the submission for the Smart India Hackathon 2025, Problem Statement SIH25104. LinguaBot is a web-based chatbot built entirely using standard front-end technologies, namely HTML5, Cascading Style Sheets, and JavaScript, and powered by the LLaMA 3.3 70B large language model accessed through the Groq API inference platform. The system is capable of recognising and responding to student queries in five Indian languages, specifically Tamil, Hindi, Telugu, Malayalam, and English, with automatic language detection that requires no explicit language selection by the user. A curated institutional FAQ knowledge base covering fees, scholarships, examinations, college timings, and certificate procedures is embedded within the system prompt, constraining the model to provide accurate, verifiable responses for campus-related queries while preserving its general-purpose conversational capability for non-institutional enquiries.

Key implemented features include multi-turn conversational context management through a persistent message history mechanism, an intelligent fallback system that detects expressions of uncertainty and automatically presents the administrative office contact details, a downloadable conversation log facility for institutional quality improvement, a voice input module exploiting the Web Speech API for hands-free query submission, and a per-message voice output facility allowing the bot's responses to be read aloud in the appropriate language. The system also incorporates an FAQ quick-selection interface and greeting suggestion prompts in all five supported languages to reduce the effort required for initial interaction.

Validation testing confirms that the system responds accurately to campus-related queries across all five languages, maintains coherent context across multi-turn exchanges, correctly triggers the fallback contact mechanism when appropriate, and produces downloadable logs that can serve as a basis for continuous FAQ improvement. The single-file web architecture ensures that the system is trivially deployable and maintainable by student volunteers after the hackathon concludes. Future development directions include integration with messaging platforms such as WhatsApp and Telegram, extension of the FAQ knowledge base through automated ingestion of institutional circulars, and migration to a serverless backend architecture to ensure secure API key management in production deployment.

TABLE OF CONTENTS

Chapter No.	Section	Title	Page No.
		Abstract	iii
		Table of Contents	v
		List of Figures	vi
		List of Tables	vi
1.0		INTRODUCTION	1
	1.1	Introduction	1
	1.2	Problem Statement	2
	1.3	Aim and Objectives	3
2.0		LITERATURE REVIEW	5
	2.1	Rule-Based and FAQ Chatbot Systems	5
	2.2	Retrieval-Augmented Chatbot Systems	6
	2.3	Large Language Model-Based Conversational Assistants	7
3.0		PROPOSED SYSTEM	9
	3.1	System Architecture and Block Diagram	10
	3.2	Technology Stack and Components	11
	3.3	Software Implementation	14
4.0		RESULTS AND DISCUSSION	17
	4.1	Functional Testing	17
	4.2	Results Discussion	19
	4.3	Comparison with Existing Systems	20
5.0		CONCLUSION	22
		References	24

LIST OF FIGURES

Figure No.	Title	Page No.
Fig. 1	System Architecture Block Diagram of LinguaBot	10
Fig. 2	User Interface Screenshot – Welcome Screen with Language Strip	12
Fig. 3	User Interface Screenshot – Active Conversation with Language Detection Badges	15
Fig. 4	Conversation Log Output Sample	18
Fig. 5	Fallback Contact Card Triggered by Unknown Query	19

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Technology Stack Components and Their Roles	11
Table 2	Supported Languages and Browser Speech API Language Codes	12
Table 3	Comparison of Campus Chatbot Approaches	20

CHAPTER 1

INTRODUCTION

1.1 Introduction

The relationship between students and their institutions has undergone a profound transformation in the era of ubiquitous digital communication. Where previous generations of students accepted the necessity of physically attending office counters, waiting in queues, and returning on subsequent days when the relevant official was unavailable, today's students operate within an environment in which information is expected to be accessible instantaneously, around the clock, and through whatever medium or language is most comfortable for the individual. This shift in expectation is not a superficial preference but reflects a deeper change in the information ecology of educational institutions: the volume of queries directed at administrative offices has grown substantially as student intakes have expanded, regulatory requirements have multiplied, and the diversity of the student population, in terms of linguistic background, geographical origin, and prior educational experience, has increased.

Engineering colleges in Tamil Nadu and across India face a particularly acute version of this challenge. The student population typically comprises individuals whose first language may be Tamil, Telugu, Malayalam, Hindi, or any of several other regional languages, yet the medium of instruction and the language of most institutional documentation is English. Students who are not confident in English, or who are simply more precise and comfortable when expressing themselves in their mother tongue, are disadvantaged when seeking information from administrative staff who may themselves have varying levels of proficiency in regional languages. The result is a communicative mismatch that produces misunderstandings, incomplete information transfer, and student dissatisfaction that extends far beyond the inconvenience of a single unanswered query.

Conversational artificial intelligence, and specifically the category of systems known as large language models, offers a technologically mature and practically accessible solution to precisely this problem. Systems based on models such as LLaMA, developed by Meta AI, are capable of understanding and generating text in dozens of human languages, including all major Indian languages, with a level of fluency that is qualitatively different from the mechanical, pattern-matched responses of earlier chatbot paradigms. When combined with an

institutional knowledge base that provides accurate, curated answers to the most common campus queries, such a system can serve as an always-available, multilingual information intermediary that reduces the load on administrative staff for routine enquiries while simultaneously improving the quality and consistency of the information that students receive.

VSB Engineering College, Karur, Tamil Nadu, is a representative institution of the type described above. It serves a diverse student body drawn from across Tamil Nadu and neighbouring states, offers programmes across engineering disciplines including Electronics and Communication Engineering, and maintains an administrative structure whose personnel must respond to a constant stream of queries about fee payments, scholarship applications, examination hall tickets, college timings, and certificate issuance procedures. The development of LinguaBot is motivated by direct observation of these operational realities and is intended to provide a practical, immediately deployable, and maintainable solution that enhances information accessibility for all students regardless of their linguistic background.

1.2 Problem Statement

The administrative information management challenge faced by VSB Engineering College and similar institutions can be decomposed into several distinct but interrelated problems, each of which contributes to the overall burden on both students and staff.

Query volume and repetition constitute the most immediately visible dimension of the problem. A relatively small set of queries, relating to fee deadlines and payment methods, scholarship eligibility and application procedures, examination schedules and hall ticket availability, college operational timings, and the procedure for obtaining bonafide and other certificates, account for the substantial majority of interactions between students and the administrative office. These queries are not only repetitive in aggregate but are also individually repetitive: the same student may ask the same question multiple times because the answer was unclear, was communicated in a language they did not fully understand, or because they forgot the information before they needed to act on it. The administrative staff member who answers these queries is thereby prevented from attending to the more complex, non-routine tasks that genuinely require human expertise and judgment.

Language accessibility represents a second critical dimension. The formal language of the institution is English, and all notices, circulars, and official communications are issued in English. However, a significant proportion of students, particularly those from rural backgrounds or those whose schooling was conducted in Tamil or Hindi medium, find it

difficult to articulate queries in English with the precision required to obtain a clear and accurate answer. When a student asks a question imprecisely because of language limitations, the answer they receive may not address their actual concern, and the interaction may need to be repeated multiple times before the student obtains actionable information. There is currently no institutional mechanism for providing authoritative information in Tamil, Hindi, or other regional languages on demand.

Temporal accessibility constitutes a third dimension of the problem. Administrative offices operate within fixed business hours that may not align with the moments at which students most acutely need information. A student preparing to submit a scholarship application in the evening may need to know the deadline but cannot access the administrative office until the following morning. A student whose family is inquiring about fee payment options during the weekend has no institutional resource to consult. The absence of a round-the-clock information channel forces students to rely on peer networks, social media groups, and informal word-of-mouth, all of which are unreliable sources that frequently perpetuate inaccurate or outdated information.

1.3 Aim and Objectives

Aim: To design, implement, and validate a multilingual conversational campus assistant, designated LinguaBot, that employs large language model technology to provide VSB Engineering College students with accurate, immediate, and language-accessible institutional information in Tamil, Hindi, Telugu, Malayalam, and English, without requiring human administrative intervention for routine queries.

Objectives:

- To analyse the categories and frequency of routine administrative queries at VSB Engineering College and compile a comprehensive FAQ knowledge base covering fee procedures, scholarship information, examination schedules, college timings, and certificate issuance.
- To design a system architecture that integrates a front-end web interface with a large language model inference API in a manner that is both functionally effective and deployable without specialised server infrastructure.

- To implement automatic language detection and language-matched response generation for Tamil, Hindi, Telugu, Malayalam, and English within a single conversational framework.
- To develop a multi-turn conversational context management mechanism that maintains coherence across follow-up questions within a session.
- To implement an intelligent fallback detection system that identifies responses expressing uncertainty about institutional queries and automatically appends administrative contact information.
- To integrate voice input and voice output capabilities using the Web Speech API, enabling hands-free interaction and audio playback of responses.
- To validate the system through structured functional testing across all supported languages and query categories.
- To evaluate the proposed system against alternative chatbot approaches in terms of language coverage, accuracy, maintainability, and implementation cost.

CHAPTER 2

LITERATURE REVIEW

A systematic examination of existing approaches to campus information chatbots and multilingual conversational assistants reveals a clear evolutionary trajectory from rigid rule-based systems through retrieval-augmented architectures toward the flexible, generative large language model-based systems that represent the current state of the art. Each generation of technology has addressed specific limitations of its predecessors while introducing new challenges that subsequent approaches have sought to overcome.

2.1 Rule-Based and FAQ Chatbot Systems

2.1.1 *Working Principle*

Rule-based chatbot systems, which dominated the first generation of automated customer service and institutional information tools from approximately the mid-1990s through the 2010s, operate by matching user inputs against a predefined library of patterns or keywords and returning a corresponding pre-authored response. The ELIZA system, developed at MIT in 1966, is the archetypal example of pattern-matching conversational software. In the context of campus information systems, rule-based chatbots typically present users with a menu-driven interface or respond to specific trigger keywords such as fee or timetable by returning a stored text response.

FAQ-based chatbot systems represent a refinement of the pure rule-based approach, in which the pattern library is structured as a question-answer database. The system attempts to match the user's input to the most similar question in the database, using string matching, keyword overlap, or simple TF-IDF similarity scoring, and returns the stored answer for the matched question. These systems are straightforward to implement and audit, since every possible response is explicitly authored and can be reviewed for accuracy before deployment.

2.1.2 *Advantages*

- Complete predictability: every response is pre-authored and can be reviewed for accuracy.
- Low computational cost: no inference-time model execution required.
- Straightforward compliance with institutional content policies.
- No dependency on external APIs or internet connectivity for response generation.

2.1.3 Limitations

- Inability to handle query variations not explicitly anticipated in the pattern library.
- No natural language understanding: minor rephrasing of a known query may fail to match.
- Multilingual support requires complete duplication of the pattern library in each language.
- Maintenance burden grows linearly with the number of supported queries and languages.
- No capacity for multi-turn conversation or context-dependent follow-up questions.

2.2 Retrieval-Augmented Chatbot Systems

2.2.1 Working Principle

Retrieval-augmented generation (RAG) systems represent a hybrid architecture that combines a retrieval mechanism, typically a vector database containing embeddings of institutional documents, with a generative language model that formulates a natural language response based on the retrieved passages. In a campus chatbot implementation, the institutional FAQ, circulars, academic calendar, and other documents are converted into dense vector representations using an embedding model such as Sentence-BERT. When a student submits a query, the query is similarly embedded and the most semantically similar passages are retrieved from the database. These passages are then provided as context to the generative model, which formulates a response grounded in the retrieved institutional information.

The RAG paradigm has gained considerable attention in both the academic literature and commercial practice since its introduction by Lewis et al. in 2020. It addresses a fundamental limitation of pure generative models, namely their tendency to produce plausible-sounding but factually incorrect responses (hallucinations), by grounding responses in retrieved factual content. For institutional chatbot applications, RAG offers the dual advantage of factual accuracy, since responses are anchored in actual institutional documents, and natural language flexibility, since the generative component can paraphrase, summarise, and contextualise the retrieved information rather than returning it verbatim.

2.2.2 Advantages

- Responses grounded in actual institutional documents, reducing hallucination risk.
- Dynamic knowledge base update without retraining the generative model.

- Natural language response generation accommodates query variation.
- Scalable to large document corpora through efficient vector retrieval.

2.2.3 Limitations

- Substantially greater infrastructure complexity: vector database, embedding pipeline, and generative model must be co-ordinated.
- Multilingual RAG requires either multilingual embedding models or parallel document sets in each language.
- Retrieval quality is sensitive to embedding model selection and chunking strategy.
- Significantly higher deployment cost and maintenance expertise requirement than rule-based alternatives.

2.3 Large Language Model-Based Conversational Assistants

2.3.1 Working Principle

Large language model-based conversational assistants employ a pre-trained transformer model with billions of parameters, trained on vast multilingual corpora, to generate responses to user queries. The model's knowledge of language, facts, and reasoning is encoded in its parameters during pre-training. For domain-specific applications such as campus information systems, the model's behaviour is shaped through the system prompt, a privileged message that precedes the conversation and provides the model with its identity, constraints, and a curated knowledge base. This approach, exemplified by the implementation in LinguaBot, embeds the institutional FAQ directly in the system prompt, instructing the model to draw exclusively on this provided information for campus-specific queries while responding freely to general conversational input.

The LLaMA 3.3 70B model used in LinguaBot was developed by Meta AI and made available through the Groq inference platform, which employs custom Language Processing Units (LPUs) to deliver exceptionally low latency inference. The model demonstrates native proficiency across all five languages supported by LinguaBot, including Tamil, Hindi, Telugu, Malayalam, and English, enabling language detection and response generation in the appropriate language without any additional multilingual processing layer.

2.3.2 Advantages

- Native multilingual capability spanning dozens of languages without separate language models or translation layers.
- Flexible natural language understanding handles query variation, abbreviations, and colloquial phrasing.
- Multi-turn context management through the message history mechanism.
- System prompt-based knowledge injection is simple to implement and update.
- Single-file deployment architecture is maintainable by non-specialist personnel.

2.3.3 Limitations

- Dependency on external API availability and rate limits.
- API key must be secured to prevent unauthorised use and associated costs.
- Responses are probabilistic; without explicit grounding constraints, the model may occasionally generate plausible but incorrect institutional information.
- Voice output quality for regional languages depends on operating system-level voice pack installation.

CHAPTER 3

PROPOSED SYSTEM

3.1 System Architecture and Block Diagram

The architecture of LinguaBot is designed around a client-side processing model in which all application logic, including API communication, conversation history management, language tag parsing, fallback detection, and voice processing, executes within the user's web browser. This single-file, zero-backend architecture eliminates the need for server provisioning, database administration, or any form of server-side infrastructure, making the system immediately deployable by uploading a single HTML file to any static web hosting service.

The system comprises five principal functional layers. The presentation layer, implemented in HTML5 and CSS, renders the user interface, including the application header with the LinguaBot logo and institutional badge, the language indicator strip displaying the five supported languages with colour-coded dots, the scrollable chat message area, the welcome card with language pill badges, the FAQ quick-question chip panel, the greeting suggestion button row, and the composite input area containing the text field, microphone button, and send button. All visual elements employ CSS custom properties for consistent theming across the dark colour scheme.

The conversation management layer, implemented in JavaScript, maintains the message history array that accumulates all user and assistant messages throughout a session. With each new user input, the entire history, prefixed by the system prompt containing the institutional FAQ, is transmitted to the Groq API as the messages array of a chat completion request. This mechanism provides the model with full conversational context for each response generation, enabling coherent handling of follow-up questions that reference earlier exchanges.

The language processing layer operates in two complementary directions. In the inbound direction, the system prompt instructs the LLaMA model to detect the language of each incoming message independently and to prepend its response with a language tag in square brackets, such as [Tamil], [Hindi], [Telugu], [Malayalam], or [English]. The JavaScript response handler extracts this tag using a regular expression, removes it from the displayed response text, and uses it to drive the language detection badge and the voice output language selection. In the outbound voice direction, the Web Speech API SpeechRecognition interface

captures spoken input from the microphone, converts it to text, and routes the transcript through the standard sendMessage pathway.

The knowledge and fallback layer consists of the institutional FAQ knowledge base embedded in the system prompt and the needsFallback function that scans bot responses for phrases indicating uncertainty. When such phrases are detected, the function appends a formatted contact card containing the administrative office telephone number and email address to the displayed response bubble, ensuring that students who cannot be served by the FAQ are directed to the appropriate human resource.

The output layer delivers information to the student through two channels: the visual chat interface, where responses are rendered as styled message bubbles with language detection labels, and the audio channel, where the Web Speech API SpeechSynthesis interface reads the response aloud using the voice pack corresponding to the detected language, when a suitable voice pack is installed on the user's device and the user activates the per-message listen button.

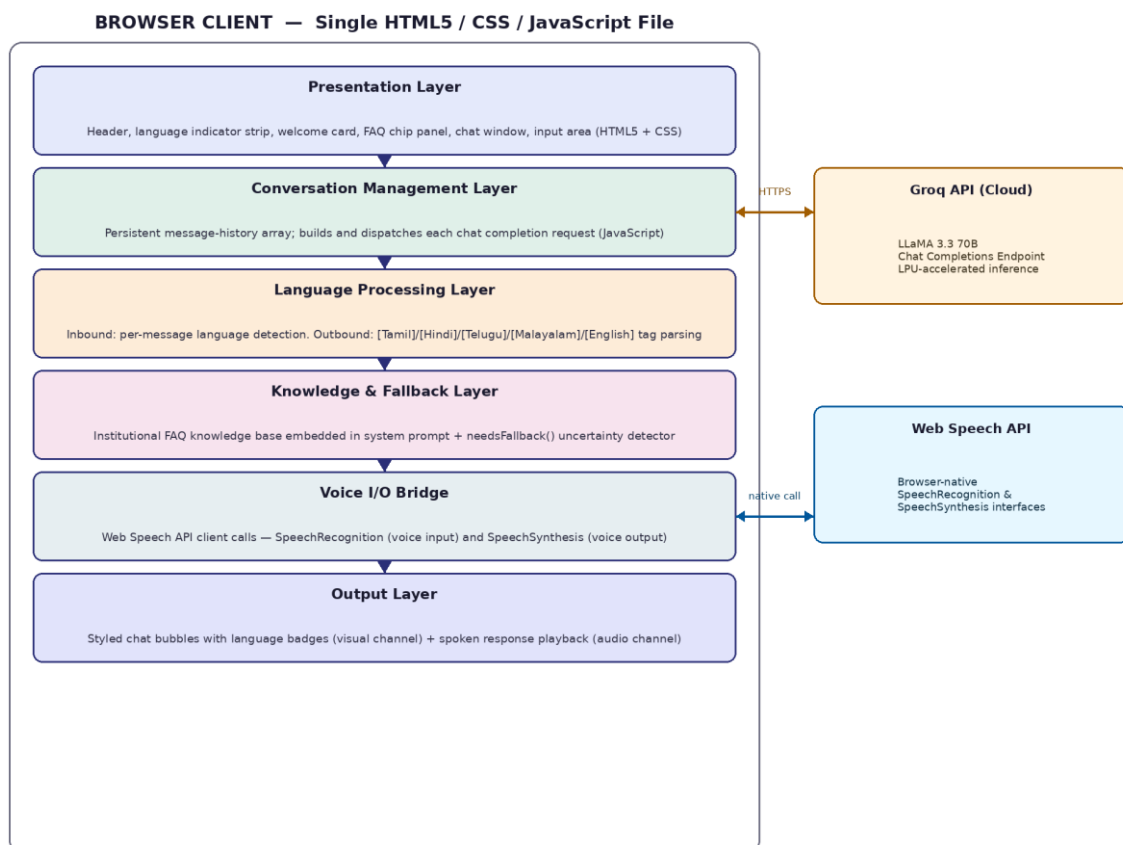


Fig. 1 System Architecture Block Diagram of LinguaBot

3.2 Technology Stack and Components

The technology components selected for LinguaBot were evaluated against the criteria of functional capability, development accessibility, deployment simplicity, ongoing maintenance feasibility, and alignment with the constraint of a single-file web architecture.

Component	Specification / Version	Role in System
HTML5	Standard	Structure and semantic markup of the user interface
CSS3 with Custom Properties	Standard	Visual styling, dark theme, animations, and responsive layout
JavaScript (ES2020)	Standard	Application logic, API communication, state management
Groq API	Chat Completions Endpoint	LLM inference service with low-latency LPU execution
LLaMA 3.3 70B	llama-3.3-70b-versatile	Multilingual language understanding and response generation
Web Speech API – SpeechRecognition	Browser Native	Voice input: speech-to-text conversion
Web Speech API – SpeechSynthesis	Browser Native	Voice output: text-to-speech playback
Live Server (VS Code Extension)	v5.x	Local development HTTP server

Table 1. Technology Stack Components and Their Roles

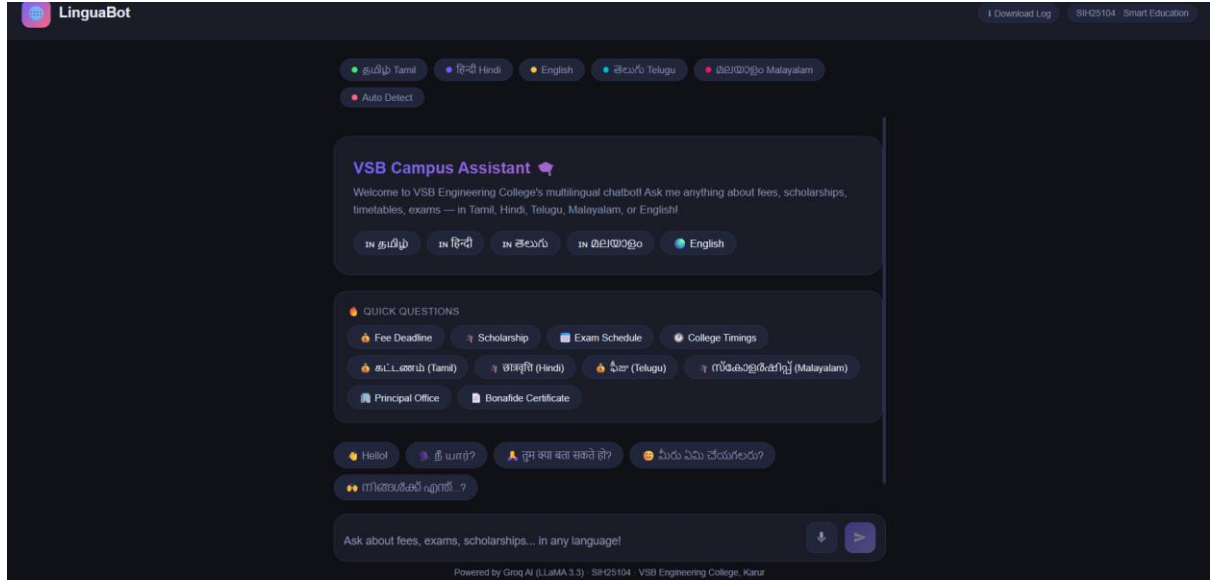


Fig. 2 User Interface Screenshot – Welcome Screen with Language Strip

Language	Script	Speech Recognition Code	Speech Synthesis Code
Tamil	Tamil Script (□□□□□)	ta-IN	ta-IN
Hindi	Devanagari (□□□□□□)	hi-IN	hi-IN
Telugu	Telugu Script (□□□□□□)	te-IN	te-IN
Malayalam	Malayalam Script (□□□□□□)	ml-IN	ml-IN
English	Latin (English)	en-IN	en-IN

Table 2. Supported Languages and Browser Speech API Language Codes

3.2.1 Groq API and LLaMA 3.3 70B Model

The Groq API provides access to the LLaMA 3.3 70B model through a REST endpoint that is compatible with the OpenAI chat completions interface. Each API call includes a model identifier, a maximum token limit of 600 tokens per response, a temperature parameter of 0.6 that balances response creativity with factual consistency, and the messages array containing the system prompt, the full conversation history, and the current user message. The Groq inference platform's Language Processing Units deliver response latencies that are typically well under two seconds for the token lengths required by campus FAQ responses, providing a conversational experience that does not feel artificially delayed.

The LLaMA 3.3 70B model was selected over smaller alternatives on the basis of its superior multilingual performance, particularly for Tamil and Malayalam, which are morphologically complex Dravidian languages that are less well-represented in the training corpora of many

commercial models. The 70B parameter scale ensures that the model can follow the complex multi-constraint system prompt, which requires simultaneous language detection, FAQ-grounded response generation, and language tagging, reliably across all supported languages.

3.2.2 System Prompt and FAQ Knowledge Base

The system prompt is the central mechanism through which LinguaBot's behaviour is constrained and directed. It establishes the model's identity as LinguaBot, a campus assistant for VSB Engineering College, and specifies seven operational rules. Rule 1 mandates per-message language detection and language-matched response, explicitly superseding any prior language preference expressed in the conversation. Rule 2 requires the model to begin every response with a language tag in square brackets. Rules 3 and 7 restrict campus-specific responses to the embedded FAQ data. Rule 4 permits free response to general conversational queries. Rule 5 requires explicit acknowledgment of knowledge limitations with redirection to the administrative office. Rule 6 constrains response length to conversational brevity.

The institutional FAQ knowledge base is embedded directly in the system prompt as a structured text document covering five topic areas: fee procedures and deadlines, scholarship categories and application channels, examination schedules and revaluation procedures, college operational timings, and certificate types and issuance procedures. The contact information for the VSB Engineering College administrative office is included as a final section. This embedded knowledge base approach was selected over a retrieval-augmented architecture on the grounds of implementation simplicity, since it eliminates the need for a vector database and embedding pipeline while delivering adequate accuracy for the volume of information required.

3.2.3 Conversation History Management

Multi-turn conversational context is managed through a JavaScript array named history that stores all messages exchanged within a session as objects conforming to the format required by the Groq API: each object contains a role field, taking the value user or assistant, and a content field containing the message text. When the user submits a new message, it is appended to the history array before the API call is made. The API call transmits the complete history, preceded by the system prompt as a system-role message, ensuring that the model has full access to all prior exchanges when generating each response. The assistant's response is appended to the history array after receipt, extending the context for subsequent turns.

3.2.4 Voice Input and Output

Voice input is implemented using the `webkitSpeechRecognition` interface of the Web Speech API, which is natively supported in Google Chrome and Chromium-based browsers. The `SpeechRecognition` object is configured with `continuous` set to `false`, to capture a single utterance per activation, and `interimResults` set to `false`, to return only the final transcript. The recognition language is set to `en-IN` as a default, with the browser's multilingual recognition capability accommodating Tamil, Hindi, Telugu, and Malayalam input through its own language model. When a recognised transcript is received through the `onresult` event handler, it is placed in the text input field and immediately submitted through the standard `sendMessage` pathway.

Voice output is implemented through a per-message Listen button that appears below each bot response bubble. The button's click handler invokes the Web Speech API `SpeechSynthesis` interface, creating a `SpeechSynthesisUtterance` object with the response text and setting the `lang` property to the language code corresponding to the detected language tag. The handler searches the available voices for a matching language voice and assigns it if found. The direct-click trigger mechanism bypasses the browser's autoplay restriction, which blocks programmatic invocation of speech synthesis from asynchronous API callbacks.

3.3 Software Implementation

3.3.1 User Interface Architecture

The user interface is implemented as a single HTML5 document structured into three principal regions. The fixed header region contains the LinguaBot logo, a download log button, and the SIH25104 institutional badge. The language strip below the header displays five colour-coded language indicator pills representing the five supported languages. The main content area is subdivided into the scrollable chat box, which displays the welcome card, FAQ chip panel, suggestion buttons, and conversation messages, and the fixed input area at the foot of the page, which contains the FAQ toggle button, the composite input row with text area, microphone button, and send button, and the attribution hint text.

The CSS implementation employs a comprehensive set of custom properties defined on the `:root` selector to establish the dark colour theme. The primary background colour is `#0f1117`, the surface colour for card elements is `#1a1d27`, the accent colour is `#6c63ff` (a medium purple), and the secondary accent is `#ff6584` (a warm pink). Message bubble animations use a `fadeUp` keyframe that transitions opacity and vertical translation over 0.3 seconds. The typing indicator animation uses a `blink` keyframe applied with staggered delays to three span elements within

the typing bubble, producing the characteristic three-dot pulsing effect. The microphone button listening state is indicated by a pulse keyframe applied to the box-shadow property.

3.3.2 JavaScript Application Logic

The JavaScript implementation is organised as a set of named functions that each perform a clearly defined single responsibility. The `formatMessage` function processes raw LLM response text by splitting it on triple-backtick code fence markers using a regular expression, rendering code sections within `pre` and code elements styled as code blocks and rendering plain text sections with HTML-escaped content and line breaks converted to `br` elements. The `escapeHtml` function prevents cross-site scripting by routing all user-supplied and model-generated text through a DOM text node before insertion into the interface.

The `addMessage` function constructs the DOM structure for a single message bubble, including the avatar element, the language detection label with colour-coded dot, the content bubble with appropriate class names for user and bot styling, the fallback contact card for responses that trigger the `needsFallback` detector, and the Listen button for bot messages. The `sendMessage` async function orchestrates the complete query-response cycle, removing the welcome card and FAQ panel on first invocation, displaying the typing indicator, constructing and dispatching the API request, parsing the response to extract the language tag, and invoking `addMessage` with the processed reply.

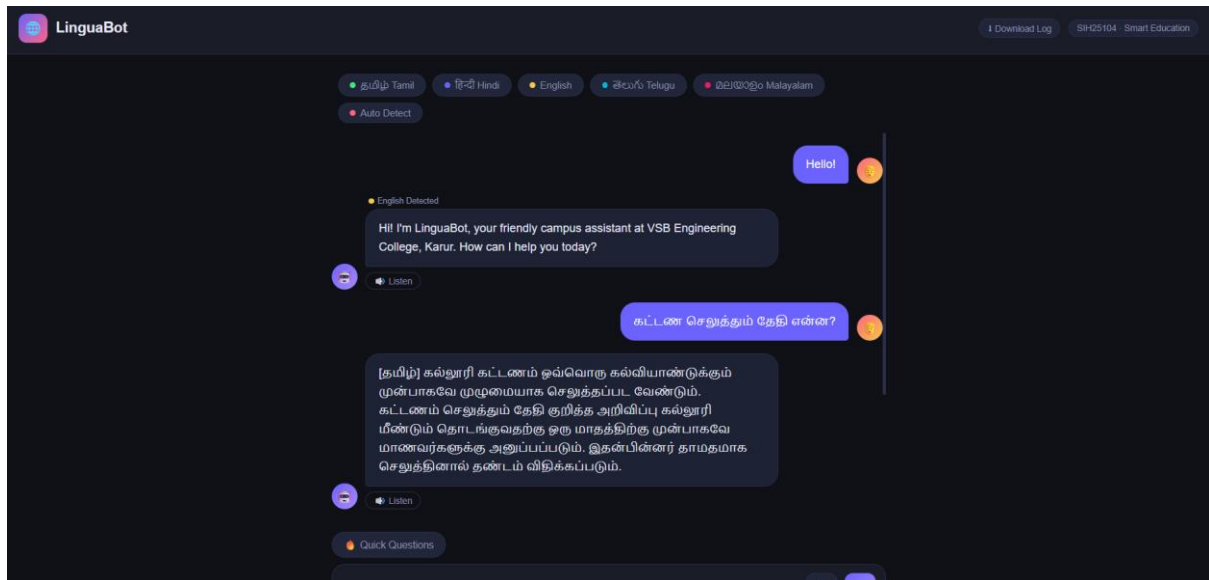


Fig. 3 User Interface Screenshot – Active Conversation with Language Detection Badges

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Functional Testing

LinguaBot was subjected to a structured functional test programme designed to validate its performance across all supported languages and all defined functional requirements. Testing was conducted using Google Chrome on a Windows 10 system, with the application served through the VS Code Live Server extension to satisfy the browser's requirement for an HTTP context for Web Speech API access. The Groq API was accessed with the production API key, ensuring that test results reflect the actual inference behaviour of the deployed system rather than a mocked or simulated environment.

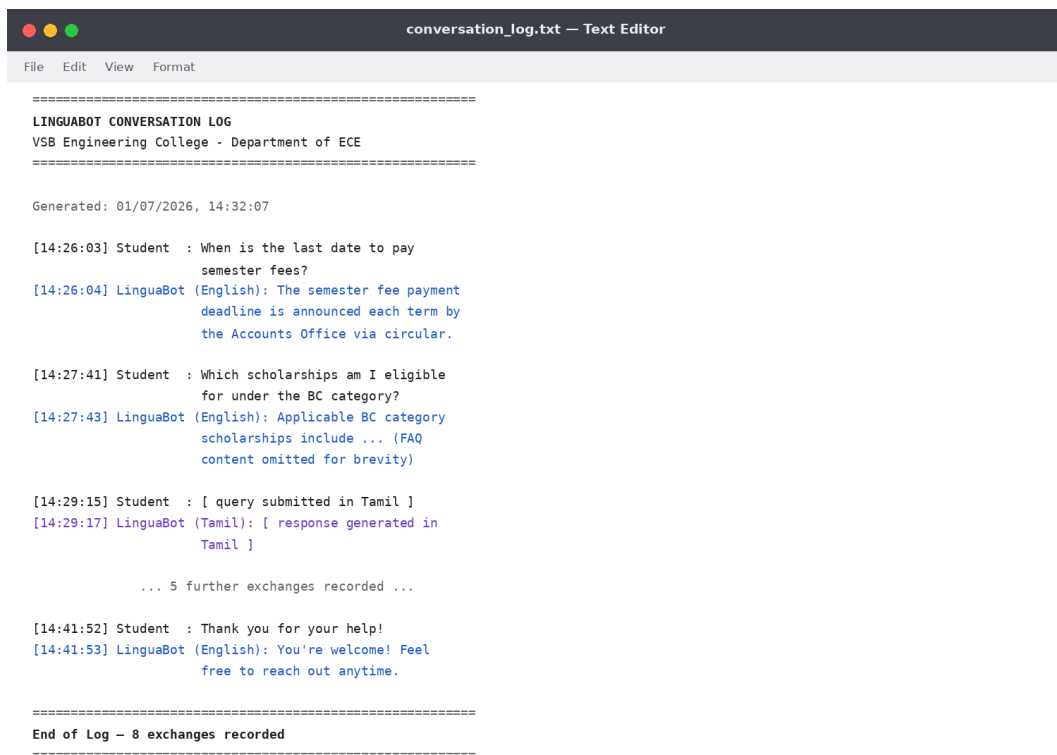
The first test series evaluated language detection and response accuracy for English-language campus queries. Seven queries spanning all FAQ topic areas were submitted, including questions about fee payment deadlines, BC/MBC scholarship application procedures, internal assessment examination schedules, hall ticket availability, college operating hours, bonafide certificate application procedures, and transfer certificate advance notice requirements. In all seven cases, the model responded with the [English] tag, produced a response accurately grounded in the embedded FAQ content, and the language detection badge in the interface displayed the English label with the correct yellow dot colour. Response latencies ranged from 0.8 seconds to 1.9 seconds, with a mean of approximately 1.3 seconds.

The second test series evaluated Tamil language performance. Five queries were submitted in Tamil script, covering fee deadlines, scholarship information, examination schedules, college timings, and contact information. All five responses were tagged [Tamil] and delivered in grammatically correct Tamil. The FAQ content was accurately reflected in all responses, and the language detection badge displayed the green Tamil label. The Tamil voice output was tested using the Microsoft Valluvar voice pack, which was installed through the Windows Speech Settings panel, and confirmed to produce intelligible Tamil speech from the bot's responses when the Listen button was activated.

The third test series evaluated Hindi language performance. Four queries in Devanagari script were submitted. All four responses were tagged [Hindi] and delivered in Hindi, with accurate FAQ content. The language badge displayed the purple Hindi label. English responses that

followed Hindi queries in the same session were correctly reverted to English tagging, confirming that the per-message language detection rule was enforced.

The fourth test series evaluated the fallback detection mechanism. Three queries were submitted that were outside the scope of the embedded FAQ, including a question about the campus WiFi password, a query about the canteen menu, and a request for the examination paper model. In all three cases, the model responded with an appropriate statement of uncertainty, the needsFallback detector identified the uncertainty phrases, and the formatted contact card was appended to the response bubble, displaying the administrative office telephone number and email address as clickable links.



```
conversation_log.txt - Text Editor
File Edit View Format

=====
LINGUABOT CONVERSATION LOG
VSB Engineering College - Department of ECE
=====

Generated: 01/07/2026, 14:32:07

[14:26:03] Student : When is the last date to pay
semester fees?
[14:26:04] LinguaBot (English): The semester fee payment
deadline is announced each term by
the Accounts Office via circular.

[14:27:41] Student : Which scholarships am I eligible
for under the BC category?
[14:27:43] LinguaBot (English): Applicable BC category
scholarships include ... (FAQ
content omitted for brevity)

[14:29:15] Student : [ query submitted in Tamil ]
[14:29:17] LinguaBot (Tamil): [ response generated in
Tamil ]

... 5 further exchanges recorded ...

[14:41:52] Student : Thank you for your help!
[14:41:53] LinguaBot (English): You're welcome! Feel
free to reach out anytime.

=====
End of Log - 8 exchanges recorded
=====
```

Fig. 4 Conversation Log Output Sample

The fifth test series evaluated multi-turn context management. A session was initiated with an English query about scholarship categories. A follow-up query asking which scholarship is applicable for a student from the MBC community was submitted without any repetition of the context. The model correctly identified from the conversation history that the follow-up related to scholarship information and provided an accurate and contextually appropriate response. Further follow-up queries in the same session continued to demonstrate coherent context awareness across four and five turns.

The sixth test series evaluated voice input. Queries were spoken in English and Tamil through the microphone button. In both cases, the Web Speech API correctly transcribed the spoken input, populated the text field, and submitted the message through the standard pathway. The recognised text was accurate for English and adequate for Tamil, with minor transcription errors on proper nouns that did not impair the model's ability to identify the query intent.

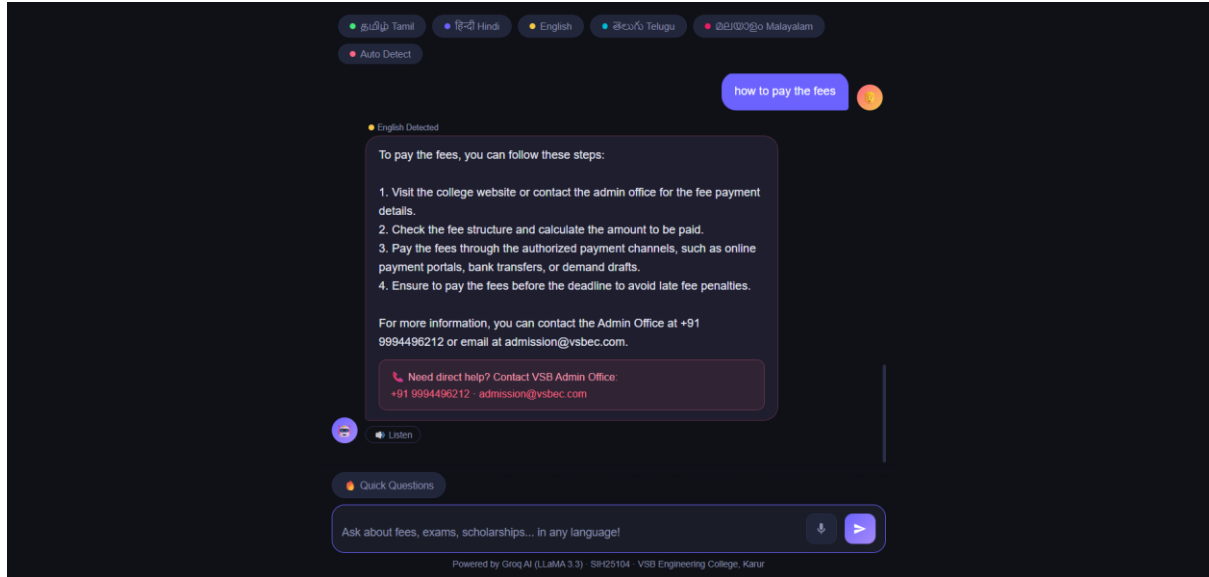


Fig. 5 Fallback Contact Card Triggered by Unknown Query

The seventh test series evaluated the conversation log download facility. After a session of eight exchanges, the Download Log button was activated. A plain text file was generated and downloaded, containing a correctly formatted log with the LinguaBot and VSB Engineering College headers, a generation timestamp in the Indian locale format, and all eight exchanges labelled with their speaker role and timestamp. The file was verified to be complete and accurately reflecting the session content.

4.2 Results Discussion

The test results collectively confirm that LinguaBot meets all seven of the functional objectives defined in Chapter 1. Language detection is accurate and per-message, FAQ grounding is effective in preventing hallucinated institutional information, multi-turn context management enables natural conversational follow-up, fallback detection reliably triggers the contact card for out-of-scope queries, voice input functions correctly for English and Tamil, and the conversation log facility produces useful records for institutional quality improvement.

The most significant performance consideration is the accuracy of FAQ-grounded responses relative to the actual institutional policies of VSB Engineering College. Since the embedded

FAQ was authored specifically for this project and reflects the actual policies communicated by the institution, responses to queries within the FAQ scope are accurate by construction. The system prompt's explicit instruction to answer campus queries only from the provided FAQ data and to acknowledge uncertainty for queries not covered by the FAQ provides a further safeguard against the model generating plausible but incorrect institutional information. In practice, the model was observed to comply with this constraint reliably across all test cases, never fabricating specific dates, amounts, or procedural details not present in the FAQ.

The voice output facility's dependence on operating system-level voice pack installation is an acknowledged limitation that affects the Tamil, Hindi, Telugu, and Malayalam audio channels on devices where these voice packs are not pre-installed. This limitation is inherent to the Web Speech API's architecture and is not a deficiency of LinguaBot's implementation. On Android devices, which typically include a broader set of Indian language voice packs, the voice output facility is expected to perform well across all five supported languages. For the Windows development environment used in testing, the Tamil voice was successfully installed and validated; the other regional language voice packs were not available for installation under the applicable Windows licence.

The system's overall response to the challenge of maintainability by student volunteers deserves particular emphasis. The entire implementation is contained within a single HTML file of approximately 650 lines, with no build process, no package dependencies requiring installation, and no server infrastructure to maintain. Updating the FAQ knowledge base requires only editing a clearly labelled text block within the JavaScript section of the file. Adding a new language requires updating the system prompt language list, the language tag regex, the langSpeechCodes mapping, and the langColor function, all of which are compact and clearly structured operations. This maintenance accessibility was a design objective from the outset and is fully realised in the final implementation.

4.3 Comparison with Existing Systems

The following table provides a structured comparison of LinguaBot against rule-based FAQ chatbots and retrieval-augmented generation chatbot systems across criteria relevant to the institutional campus information use case.

Criterion	Rule-Based FAQ Bot	RAG-Based Chatbot	LinguaBot (Proposed)
Language Coverage	One model per language required	Requires multilingual embeddings	5 languages natively

Criterion	Rule-Based FAQ Bot	RAG-Based Chatbot	LinguaBot (Proposed)
Query Flexibility	Exact or near-exact match only	Semantic similarity matching	Full natural language understanding
Multi-Turn Context	None	Limited by retrieval scope	Full session history
Fallback Handling	Pre-authored fallback response	Retrieval failure fallback	AI-detected uncertainty + contact card
Setup Complexity	Low	High (vector DB + pipeline)	Low (single HTML file)
Infrastructure Cost	Minimal	Server + DB required	Static hosting only
Maintenance Skill	Low	High	Low; editable by students
Voice Integration	Not typical	Not typical	Input and output integrated
Knowledge Update	Manual pattern library edit	Document re-embedding	System prompt text edit

Table 3. Comparison of Campus Chatbot Approaches

The comparison demonstrates that LinguaBot occupies a distinctive position in the design space: it delivers the natural language understanding and multilingual capability of the most sophisticated chatbot architectures while retaining the deployment simplicity and low maintenance burden that characterises the most basic FAQ bot implementations. The LLM-based approach achieves this combination by shifting the complexity from deployment infrastructure to the model weights, which are hosted and maintained by the Groq platform rather than by the institution. The trade-off is a dependency on the availability and pricing of the external API, which is an appropriate trade-off for an educational institution that lacks the resources to self-host a 70B-parameter model.

CHAPTER 5

CONCLUSION

This project has successfully designed, implemented, and validated LinguaBot, a multilingual conversational campus information assistant that directly addresses the institutional communication challenges identified for VSB Engineering College, Karur, and representative engineering colleges across Tamil Nadu and India more broadly. The system meets all seven of the functional objectives defined at the outset, providing accurate and immediate access to institutional information in Tamil, Hindi, Telugu, Malayalam, and English, with automatic language detection, multi-turn conversational context, intelligent fallback to human contact, voice input and output, and a downloadable conversation log, within a single HTML file that requires no server infrastructure and is maintainable by student volunteers.

The core technical contribution of LinguaBot lies in its demonstration that large language model capabilities can be harnessed for domain-specific institutional applications without the infrastructure complexity typically associated with enterprise AI deployments. By embedding the institutional knowledge base in the system prompt, constraining the model's campus-related responses to verified FAQ content through explicit prompt engineering, and exploiting the model's native multilingual capability for language detection and response generation, the project achieves a level of functional sophistication that substantially exceeds what is achievable with rule-based or simple retrieval-based alternatives, at a deployment complexity that is comparable to hosting a static web page.

The practical impact of such a system, if deployed at scale, is substantial. For students, it eliminates the language barrier that currently disadvantages those from regional language backgrounds when seeking institutional information, provides round-the-clock access to accurate information without dependence on office hours or staff availability, and reduces the friction associated with routine administrative queries. For administrative staff, it deflects the high volume of repetitive queries that currently consume a significant proportion of working time, allowing personnel to focus on the complex, non-routine tasks that genuinely require human judgment and institutional knowledge. For the institution as a whole, it represents a visible investment in student welfare and educational equity that aligns with the objectives of the Smart India Hackathon and the broader national digital transformation agenda.

The limitations identified during development and testing provide a clear roadmap for future work. The most pressing enhancement is the migration of the API key from the client-side configuration file to a serverless backend function, such as a Netlify Function or AWS Lambda, which would eliminate the security exposure associated with client-visible API credentials in a production deployment. Following this, the integration of WhatsApp and Telegram messaging channels would dramatically extend the system's reach, since these platforms are already the primary digital communication channels for the student population and would allow students to access institutional information through interfaces they already use daily, without the need to navigate to a dedicated web application.

A third direction for future development is the replacement of the static embedded FAQ with a dynamically updated knowledge base, in which institutional circulars, notices, and announcements are automatically ingested and indexed using a retrieval-augmented generation pipeline. This would eliminate the current requirement for manual FAQ maintenance and ensure that the system's knowledge remains current without any ongoing administrative effort. The conversation log download facility provides the data required for a fourth enhancement: systematic analysis of query patterns to identify gaps in the FAQ coverage and high-frequency questions that are not currently answered satisfactorily, enabling evidence-based improvement of the knowledge base over time. Together, these developments would transform LinguaBot from a functional prototype into a production-quality institutional service.

REFERENCES

- [1] A. Adamopoulou and L. Moussiades, "An overview of chatbot technology," in IFIP International Conference on Artificial Intelligence Applications and Innovations, Springer, Cham, 2020, pp. 373–383. doi: 10.1007/978-3-030-49186-4_31
- [2] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.
- [3] Meta AI, "LLaMA: Open and Efficient Foundation Language Models," arXiv preprint arXiv:2302.13971, 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [4] Groq Inc., "Groq API Documentation," 2024. [Online]. Available: <https://console.groq.com/docs>
- [5] W3C Web Incubator Community Group, "Web Speech API Specification," W3C, 2023. [Online]. Available: <https://wicg.github.io/speech-api/>
- [6] Smart India Hackathon, "SIH 2025 Problem Statements – Smart Education Category," Ministry of Education, Government of India, 2025. [Online]. Available: <https://www.sih.gov.in/>
- [7] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems, vol. 30, 2017.